



Journal of Advanced Research in Applied Sciences and Engineering Technology

Journal homepage:
https://semarakilmu.com.my/journals/index.php/applied_sciences_eng_tech/index
ISSN: 2462-1943



Measuring Software Maintainability: An Analysis of Evolving Metrics across Development Paradigms

Sharifah Mashita Syed-Mohamad^{1,*}, Amir Ngah¹, Al-Fahim Mubarak Ali², Pantea Keikhosrokiani³

¹ Faculty of Computer Science and Mathematics, Universiti Malaysia Terengganu, 21030 Kuala Nerus, Terengganu, Malaysia

² Faculty of Computing, Universiti Malaysia Pahang Al-Sultan Abdullah, 26600 Pekan, Pahang

³ Empirical Software Engineering in Software, Systems, and Services, University of Oulu, 90014 Oulu, Finland

ARTICLE INFO

ABSTRACT

Keywords:

Software maintainability; software metrics; Agile; DevOps; continuous maintenance; continuous integration; evolution

Maintainability is a critical aspect of modern software projects, referring to the ease with which a system can be modified, repaired, or adapted to changing requirements. In today's agile environment, maintainability is becoming increasingly important, yet there is not much literature that reports whether the metrics established since several decades ago are still used in modern projects. Are there different measures of software maintainability in Agile and DevOps, in particular? This paper reviews the evolution of software maintainability metrics from traditional function-oriented to contemporary software development approaches. The objective is to understand the functions and limitations of software metrics in each paradigm and understand the impact of continuous practices on maintainability in modern software development. It is observed that those metrics particularly the many object-oriented metrics were developed based on the assumptions and practices prevalent during the waterfall and traditional software development era. Agile, DevOps and other modern development practices have different approaches and techniques that may require different metrics to assess software maintainability effectively.

1. Introduction

Software maintenance is a crucial part of the software development process, which includes efforts targeted at preserving, upgrading, and improving software systems after they have been deployed. The importance of software maintenance in the context of software development has been highlighted in various research publications [1-4]. According to the ISO/IEC 25010, software maintainability is defined as “the degree of effectiveness and efficiency with which a product or system can be modified to improve it, correct it, or adapt it to changes in the environment and requirements” [5]. This definition emphasizes the necessity for effective metrics that can accurately assess maintainability, which is essential for evaluating software quality and guiding maintenance efforts, as it is one of the most important aspects to consider in order to avoid poor system

* Corresponding author.

E-mail address: s.mashita@umt.edu.my

performance, reliability issues, and threats to long-term viability [5]. High maintainability not only reduces maintenance costs but also extends the lifespan of software systems and facilitates the integration of new technologies. Conversely, software with poor maintainability often incurs higher costs, demands more resources, and leads to increased downtime, ultimately affecting user satisfaction and system reliability. As such, maintaining high coding standards and producing well-documented, readable code is imperative for successful software maintenance.

The rapid advancements in software development within Industry 4.0 necessitate a focus on skill enhancement to navigate dynamic, technology-driven environments [6]. These advancements highlight the evolving demands of modern software development practices. In this context, the adoption of Agile and DevOps methodologies has introduced both challenges and opportunities for software maintenance. As software development environments become more complex, particularly with the advent of Agile and DevOps methodologies, there is an increasing emphasis on the need for comprehensive quality management that extends beyond just software to include network and infrastructure performance [7]. Traditional metrics such as Cyclomatic complexity [8], Maintainability Index (MI) and lines of code [9] are increasingly insufficient in capturing the complexities of modern development environments, underscoring the need for re-evaluating existing maintainability metrics [10]. Maintaining software in the modern environment can be challenging due to the rapid pace of development and the need for frequent updates and deployments. The maintenance challenges can lead to roadblocks and hinder the effective management and improvement of software quality [10], [11-13]. To address these issues, this systematic literature review explores the evolution of software maintainability metrics across different development paradigms. By identifying and categorizing established metrics, this study aims to provide valuable insights into their applicability and effectiveness in contemporary software development contexts.

The following research questions guide this study:

- i. What are the established metrics for software maintainability across different development paradigms?
- ii. What are the continuous maintenance metrics for modern software development projects?
- iii. What are the strengths and weaknesses of different maintainability metrics in assessing software quality?
- iv. What emerging trends and future directions can be identified in the field of software maintainability metrics?

To support a sufficient lifetime analysis, this study focuses on peer-reviewed research literature, excluding grey literature, and prioritizes studies with empirically obtained results. By understanding the functions and limitations of these metrics in various development paradigms, this research aims to contribute valuable insights into their effectiveness in the current software landscape. This article is organized as follows: Section 2 focuses the literature review process and search strategy. Section 3 presents findings based on the research questions. Finally, Section 4 concludes the work and suggesting future research directions.

2. Methodology

The review analysis process follows a structured approach to identify, evaluate, and synthesize relevant literature on software maintainability metrics. This process ensures transparency,

reproducibility, and rigor in the review, allowing for a comprehensive understanding of the current state of research in this area.

2.1 Search Strategy

A comprehensive search strategy was developed to identify relevant studies. The following steps were undertaken:

- i. **Database Selection:** Key electronic databases were selected for the literature search, including IEEE Xplore, ACM Digital Library, Scopus, and Web of Science. These databases were chosen for their extensive coverage of software engineering and computer science literature.
- ii. **Search Terms:** A set of keywords and phrases was formulated based on the research questions. The search terms included "software maintainability metrics," "DevOps," "Agile methodologies," "software maintenance," and "evolution of metrics." Boolean operators (AND, OR) were used to combine these terms effectively.
- iii. **Search Execution:** The search was conducted using the selected databases, applying the defined search terms. The search results were exported for further screening.

2.2 Inclusion and Exclusion Criteria

To ensure the relevance and quality of the studies included in the review, specific inclusion and exclusion criteria were established: The inclusion criteria were as follows: peer-reviewed articles published in reputable journals or conference proceedings were considered essential. Additionally, the review focused on studies that specifically examined software maintainability metrics across various development paradigms, including traditional, Agile, and DevOps. Finally, empirical studies that provided data on the effectiveness or applicability of maintainability metrics were prioritized for inclusion.

In contrast, the exclusion criteria helped filter out less relevant literature. Non-peer-reviewed sources, such as white papers, opinion pieces, and industry reports, were not considered. Furthermore, studies not available in English were excluded to maintain consistency in data analysis. Lastly, articles that did not focus on software maintainability metrics or lacked empirical evidence were also excluded from the review.

2.3 Data Extraction and Synthesis

Data extraction was conducted to gather relevant information from the selected studies, focusing on several key points. These included study characteristics such as authors, publication year, type of study (empirical or theoretical), and research context. Additionally, specific maintainability metrics discussed in each study were identified. The key findings related to the effectiveness, applicability, and evolution of these metrics were documented, along with any research gaps that warranted further investigation.

The synthesis of the extracted data followed a qualitative approach, primarily through thematic analysis. This analysis involved coding the data to identify recurring themes and patterns related to software maintainability metrics. These themes were grouped into categories based on development paradigms and the specific metrics discussed. A comparative analysis was also performed to evaluate the strengths and weaknesses of various maintainability metrics across different paradigms. This comparative approach aimed to highlight how traditional metrics apply in modern software development contexts and identify emerging trends in maintainability.

3. Findings

3.1 Software Maintainability Metrics

Software maintainability is a crucial aspect of the software development process, as it ensures that software systems can be easily modified, updated, and improved after deployment. Highly maintainable software is easy for teams to keep in top shape with minimal time and effort. Poor quality software is harder to maintain, requires a larger effort, and costs more to adapt to new requirements. Many object-oriented metrics have been proposed in the literature dates back to the early 1990s when researchers began exploring the use of metrics to predict software maintainability in order to ensure long-term sustainability [8], [14], [15]. Since then, various software metrics and models have been developed and described to assess the maintainability of object-oriented software systems [16], [17]. There is not a single metric that can directly measure software maintainability. Instead, there are several metrics and approaches that can provide insights into maintainability-related aspects of software. Early literature has shown that, maintainability metrics are classified into three areas [14], discussed in the following sections.

3.1.1 Code-Based Metrics

Code-based metrics are software metrics that focus on analysing the characteristics and properties of the source code. Code metrics provide quantitative measurements of the software code, which can help developers and teams understand the structural complexity of the code. McCabe's cyclomatic complexity is one of the main software metrics that is widely applied in managing and controlling software complexity. It measures the number of independent paths in a software system, with a higher cyclomatic complexity score, indicates a more complex system as it contains more possible execution paths through the code [18]. Other than that, it is also often associated with higher criticality and increased defect density in the code [19] which means that the likelihood of introducing defects also increases. As a result, high cyclomatic complexity scores are often associated with lower software maintainability as they can result in code that is more error-prone, less modular and harder to test and maintain. However, there are some limitations to using McCabe's Cyclomatic Complexity, such as its inability to capture all aspects of software complexity and its lack of accuracy in predicting software maintainability. Despite these limitations, McCabe's Cyclomatic Complexity remains a valuable tool for measuring software complexity and improving software maintainability.

Code churn is another widely used metric for measuring software maintainability, which was first introduced in 2005 [20]. It is used to quantify and measure the frequency of changes in a codebase over a period of time. A higher code churn rate indicates that the system codebase is undergoing frequent changes, which could reduce system maintainability if the changes are not well-designed. However, in modern development environments, formal code metrics such as Lines of Code (LOC) and code complexity are considered less useful, and agile methodology has a unique approach to software development metrics and how to use them effectively.

3.1.2 Structure-Based Metrics

Structure metrics focus on analysing the software system's architecture and design. A well-known structure metric is Henry and Kafura's information flow metric [21]. This measurement is basically based on the amount of information which flows in and out of a procedure, as can be determined using the following formula:

$$C_p = \text{length} * (\text{fan-in} * \text{fan-out})^2,$$

- where C_p represents procedure complexity while length represents number of lines of text in source code for the procedure.
- Fan-in: The number of flows of information into a procedure including the number of global data structures from which the procedure retrieves information. It represents the incoming dependencies and inputs to a specific procedure.
- Fan-out: The number of flows of information from a procedure, including the number of global data structures that the procedure updates. It represents the outgoing dependencies and outputs from a specific procedure.

In other words, Henry and Kafura's metric focuses on the communication and data exchange between different parts of the system, specifically within a procedure. A higher procedure complexity which indicates that a procedure has a large number of information flows and dependencies, which can make it harder to understand, test and maintain.

3.1.3 Hybrid Metrics

Hybrid metrics combine both code and structure metrics to provide a comprehensive understanding of software maintainability. Examples of hybrid metrics include Maintainability Index [22], Halstead's Software Science, and Chidamber and Kemerer's Metrics Suite [23]. One commonly used set of structural metrics is the Chidamber and Kemerer (CK) metrics, which includes six metrics that quantify different quality attributes related to maintainability qualities [23]. These metrics provide valuable insights into the complexity, coupling, cohesion, and other structural aspects of the codebase, which directly impact the ease of maintaining and evolving the software over time. The subset of the CK metrics used to evaluate the system includes:

- i. Weighted Methods per Class (WMC) - WMC is the sum of the complexities of all class methods. It indicates the effort required to develop and maintain a class. The complexity can be measured using cyclomatic complexity. The larger the number of methods in a class, the greater the potential impact on children, limiting the possibility of reuse.
- ii. Response For a Class (RFC) - RFC is the count of all methods that can be potentially executed in response to a message received by an object. It measures the potential complexity of a class and its ability to respond to different messages.
- iii. Lack of Cohesion in Methods (LCOM) - LCOM measures the lack of cohesion within a class by calculating the number of pairs of methods that do not share instance variables. A higher LCOM value indicates lower cohesion and potentially higher complexity.
- iv. Coupling Between Objects (CBO) - CBO measures the number of other classes to which a class is coupled. It indicates the level of interdependence between classes. Higher CBO values suggest higher coupling and potential complexity.
- v. Depth of Inheritance Tree (DIT) - DIT measures the number of levels in the inheritance hierarchy for a class. It indicates the depth of the class in the inheritance tree. Higher DIT values suggest a higher level of complexity due to inheritance.
- vi. Number of Children (NOC) - NOC measures the number of immediate subclasses that inherit from a class. It indicates the number of child classes a class has. Higher NOC values suggest a higher level of complexity due to inheritance and potential impact on children.

Maintainability Index (MI) was introduced by Oman and Hagemeister [22], is a composite metric which combines multiple code-based metrics, including McCabe's Cyclomatic Complexity (CC), Halstead's Volume (V), and Lines of Code (LOC) to quantify the maintainability of a software product [24]. Components above the 85 MI are considered as highly maintainable, components between 65 and 85 MI are considered as moderately maintainable while components below 65 MI are considered as difficult to maintain. The Maintainability Index can be calculated using the following mathematical formula:

$$MI = 171 - 52 * \ln(\text{aveV}) - 0.23 * \text{aveV(g)} - 16.2 * \ln(\text{aveLOC}),$$

where aveV represents the average Halstead Volume, aveV(g) represents the average Cyclomatic Complexity per module, and aveLOC represents the average Lines of Code per module.

In conclusion, measuring software maintainability is a complex task, and there are several code-based, structure-based, and hybrid metrics that can provide insights into maintainability-related aspects of software. These metrics are used to assess the degree to which an application is understood, repaired, or enhanced, and they play a crucial role in identifying improvement areas. In contemporary software development environments, the significance of software maintainability metrics has surged, particularly within the context of DevOps software development, which thrives on continuous development and rapid delivery.

3.2 Continuous Practices in Modern Software Development

Software development has evolved significantly over the years, with a shift towards more iterative, collaborative and continuous approaches. While proponents of Agile methods promote customer collaboration to elicit proper software maintenance, the context of software maintenance is not always clear. Despite the use of Agile practices in software maintenance, empirical studies have identified the challenges of Agile software maintenance and the quality factors affected by it. Effective communication, continuous improvement, and proper documentation are emphasized in the article as necessary to overcome these challenges [11].

In contrast, DevOps practices such as continuous integration and deployment (CI/CD) are seen as a solution to the challenges of Agile through the concept of Continuous Maintenance. Continuous maintenance in DevOps software development refers to the practice of properly and consistently maintaining repositories and artifacts throughout the software development lifecycle [12]. DevOps emphasizes collaboration, communication, and automation between development and operations teams to enable faster and more reliable software delivery. The traditional metrics may not fully capture the complexities and dynamics of modern software systems, which are often developed in smaller increments and undergo frequent changes. Based on our research, the implementation of DevOps practices in software development has been studied extensively in the literature. We discuss the practices and relate to the application of maintainability metrics in the following sections.

3.2.1 Continuous Maintenance

Continuous maintenance (CM) gained popularity after its introduction in the 1998 book "Microsoft Secrets," which showcased Microsoft's approach to software development through a "synch-and-stabilize" process [25]. Unlike traditional software projects, Microsoft's projects were divided into multiple milestone cycles without a separate product maintenance phase. Instead, they

shipped later versions of their products that added more functions to the existing, running product. This approach implied that rapidly evolving software projects underwent continuous changes and modifications, without a definitive stabilization phase to examine code stability and reliability [26].

CM has been associated with continuous integration (CI) by many literature sources [27], [28]. CM plays a crucial role in DevOps, as it reinforces the cycle of continuous deployment, feedback, and maintenance or incident response that teams need to keep always-on services, always on [28], [29]. Automation, continuous testing, continuous improvement, collaboration and cross-functional teams, customer-centric approach are the essential elements in DevOps to ensure the software's continued operation and evolution Fig. 1 shows all practices that are essential for DevOps process execution [30]. It is reported that the culture of continuous improvement promoted by DevOps ensures that maintenance processes are constantly refined, resulting in higher efficiency and better outcomes over time [13]. In the following sections, we discuss the practices in relation to CM and metrics being used in DevOps projects that are reported in the literature.

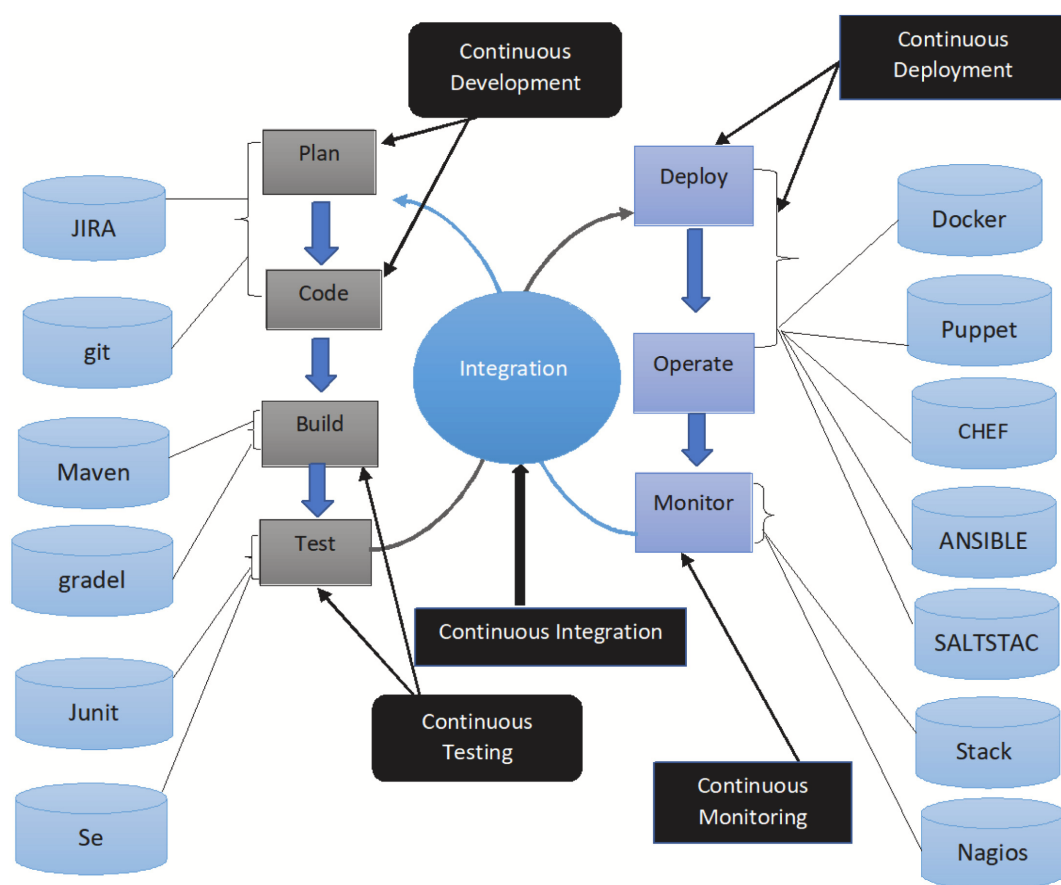


Fig. 1: DevOps life-cycle, including various phases (gray and blue boxes), tools (illustrated as cylinders), and practices. Source: Adopted from [30]

3.2.2 Continuous Development

Continuous development encompasses two essential phases: planning and coding. During this stage, project objectives are defined, and developers initiate the creation of code for the specific application. Key DevOps tools utilized in this phase include JIRA and Git, facilitating efficient collaboration and version control. Continuous development is a pivotal stage within the DevOps lifecycle, enabling teams to iteratively build and enhance software products. Metrics such as

deployment frequency play a crucial role in this phase, aiding in the early detection of integration issues and expediting defect resolution. Additionally, these metrics optimize resource allocation, allowing testers to focus on exploratory testing and value-added activities, thus enhancing overall software quality and efficiency [31]

3.2.3 Continuous Testing

Continuous testing is the stage where the developed application goes through continuous tests to detect bugs. According to [32] continuous testing has two characteristics. There could be testing for the whole life-cycle, meaning testing continuously from the beginning to the end, or strategic test automation, meaning tests are continuously adapted and executed based on the requirements. There are various automation tools to support the testing process.

3.2.4 Continuous Integration

Continuous integration is a process that combines various steps, such as code compilation, code validation coverage, testing acceptance, compliance of coding standards, and deployment package building [33]. Continuous integration initiates faster feedback for the developers by detecting integration errors. To maintain a good practice, developers need to integrate their work onto a common repository on a daily basis. Each integration should be followed by building and testing, so that the system can remain functional after new changes are introduced by developers. This iteration of integration, source code testing, and fixing problems will efficiently increase system progress [10]. This process helps to identify and address issues early on, reducing the likelihood of merge conflicts, bugs, and duplicated efforts. CI keeps the main branch up-to-date and encourages developers to continuously merge their changes into the shared repository.

3.2.5 Continuous Delivery

Continuous delivery is an essential process nowadays [13,30]. Continuous delivery is a set of principles and design practices that increases deployment frequency. As a result, this practice is welcomed by most companies, as it increases the quality, efficiency, and reliability of the product. Companies are willing to introduce continuous delivery because that decreases the cycle time for software delivery. However, when adopting continuous delivery in companies, several problems and issues arise. Specifically, these problems were found in design, testing, release, and integration activities, and others were related to the organizational, human, and resource aspects of continuous delivery [10,13].

3.2.6 Continuous Deployment

Continuous deployment is a process that helps to deploy software codes quickly and automatically by maintaining a standard of quality. During the process of continuous deployment, there is no need for any manual phases or developer decision making for any issues related to production deployment [8]. Skelton and O'Dell [42] have also suggested that continuous deployment is an approach in which automated tests are conducted while developers commit new features. Continuous deployment is a good way to gather feedback from users, which also reduces production costs.

3.2.7 Continuous Monitoring

Monitoring is another important stage in the DevOps lifecycle. According to Schlossnagle [51], monitoring is a system for observing and determining the correctness of the system's behavior. This is a phase in which the developers continuously monitor the performance of the software system. In the monitoring phase, all information related to the software application is given earlier, so that vital information can be processed quickly to recognize the application [51].

3.3 Continuous Maintenance Metrics

Continuous maintenance metrics in DevOps are used to measure the effectiveness and efficiency of the maintenance process in ensuring the availability, reliability, and performance of software systems. Previous research has identified DevOps Research and Assessment (DORA) metrics to track software development team performance which include Mean Lead Time for Changes, Mean Time to Recover, Change Failure Rate, and Deployment Frequency. However, there is limited research on whether these metrics can also indicate software maintainability. A systematic mapping study on quality measurement in agile and rapid software development found that there is a need for more research on quality metrics in DevOps environments. Empirical studies have identified the challenges of DevOps software maintenance and the quality factors affected by it [11], [30].

Continuous maintenance metrics in DevOps are essential for measuring the performance of the software development pipeline and identifying bottlenecks in the process. Here are some of the continuous maintenance metrics in DevOps:

- i. Mean Time to Failure (MTTF): MTTF is a general maintenance metric that shows the frequency of failures in the system. It measures the average time between failures and is used to identify areas for improvement in the system [1].
- ii. Server Uptime: Server uptime is the amount of time that a server is operational and available for use. It is a critical metric for measuring the reliability and availability of the system [1].
- iii. Mean Time Between Failures (MTBF): MTBF is a maintenance metric that measures the average time between system failures. It is used to identify areas for improvement in the system and to estimate the system's reliability [1].
- iv. Mean Time to Recovery (MTTR): MTTR is the average time it takes to recover from a system failure. It is a critical metric for measuring the system's resilience and the effectiveness of the maintenance process [1,3].
- v. Deployment Frequency: Deployment frequency is the number of times that changes are deployed to the production environment. It is a key metric for measuring the speed and efficiency of the deployment process [2,3].
- vi. Lead Time for Changes: Lead time for changes is the time it takes to implement a change from the time it is requested to the time it is deployed. It is a critical metric for measuring the efficiency of the change management process [2,3].
- vii. Change Failure Rate: Change failure rate is the percentage of changes that result in a failure in the production environment. It is a key metric for measuring the quality of the changes and the effectiveness of the testing process [3].
- viii. Application Performance: Application performance metrics measure the performance of the application, including response time, throughput, and error rates. These metrics are critical for measuring the quality of the application and identifying areas for improvement [5].

- ix. **Mean Time to Detection (MTTD):** MTTD is the average time it takes to detect a problem in the system. It is a critical metric for measuring the effectiveness of the monitoring and alerting process [5].

Overall, maintenance metrics in DevOps are essential for measuring the performance of the software development pipeline and identifying areas for improvement. These metrics provide critical information for DevOps teams to have visibility and control over their development pipeline and ensure the software's continued operation and evolution. However, empirical studies have identified the challenges of DevOps software maintenance and the quality factors affected by it [11,30,34]. These metrics need to be further researched as suggested by a systematic mapping study on quality measurement in agile and rapid software development.

3.4 DevOps Toolchain

DevOps continuous practices are realized through a collection of tools that operate as an integrated unit to design, build, test, manage, measure, and operate software and systems. The use of a toolchain and support in deployment pipeline activities accelerates the delivery of software changes and bug fixes, which can help in maintaining software systems [35-38]. However, there are significant challenges to DevOps tool adoption, including conflicting goals between development and operations teams and senior management's reluctance to invest in DevOps automation tools [39-41].

In summary, continuous practices like Agile and DevOps have transformed modern software development by enhancing collaboration, iterative progress, and ongoing software maintenance. Continuous Maintenance, along with DevOps practices such as continuous testing, integration, delivery, and deployment, plays a crucial role in sustaining software performance and stability. Key metrics, including Mean Time to Failure, Server Uptime, Deployment Frequency, and Application Performance, are utilized to monitor and improve these processes. However, research indicates that further understanding is needed on how these metrics specifically impact software maintainability. DevOps toolchains, which expedite software updates and bug fixes, encounter obstacles like conflicting team goals and management reluctance to invest in automation tools. The following sections will delve into how these continuous practices and metrics specifically support DevOps and their implications for advancing software development strategies.

4. Discussion: The Evolution of Software Maintainability Metrics and Future Directions

The evolution of software metrics from traditional function-oriented to object-oriented, component-based, and aspect-oriented paradigms has been a subject of exploration in the literature [17,18,42]. These attempts have aimed to understand the functions and limitations of software metrics in each paradigm and highlight their importance in assessing software quality and maintainability. Table 1 provides an overview of different software development paradigms and metrics used to assess software maintainability. As traditional function-oriented metrics lack in quality parameters like reusability, maintainability and so, object-oriented metrics have become one of the popular concepts in today's software development environment. Object-oriented metrics have been widely accepted because of many attributes like reusability, better abstraction, polymorphism, simplicity, time-saving, cost effectiveness and easy maintenance in the software code. Similarly, component-based and aspect-oriented metrics take this journey forward to achieve these quality aspects in the software more effectively [43,44].

Table 1

The evolution of maintainability metrics based on software development paradigms adapted from [42]

Paradigms	Description	Limitation	Metric Example
Traditional function-oriented	- Provide quantitative measures of software size and complexity. - These metrics aid in estimating effort, resource allocation, and project planning [22].	- Metrics based solely on lines of code may not capture the true complexity or quality of the software. - They may not adequately address the modularity and maintainability of the code.	Lines of code (LOC), function points, McCabe's Cyclomatic Metric.
Object-oriented	- Object-oriented metrics capture the complexity and maintainability of object-oriented systems. - These metrics facilitate better design decisions, code reusability, and modularization [23].	- Object-oriented metrics may not fully capture the dynamic nature of object-oriented systems. - They may not effectively measure the quality of interactions between objects or the impact of design patterns.	Weighted methods per class (WMC), depth of inheritance tree (DIT).
Component-based	- Metrics in the component-based paradigm emphasis on software component assembly. - These metrics aid in assessing the impact of component changes and promoting component-based development practices [43].	- Metrics in the component-based paradigm may not consider the context-specific requirements of individual components. - They may not address the challenges of component integration and interoperability.	Average Interaction Density (AID), Incoming Interaction Density (IID), Outgoing Interaction Density (OID), Average Interaction Density (AID)
Aspect-oriented	- Aspect-oriented metrics focus on capturing the modularity and maintainability of cross-cutting concerns. - These metrics aid in managing the complexity of aspect-oriented systems to ensure a good modularization [44].	- Aspect-oriented metrics may not provide a comprehensive assessment of the overall system's maintainability. - They may not effectively capture the interactions between aspects and other system components.	Degree of crosscutting per pointcut (DCP), number of advice per aspect (NAA), number of pointcuts per aspect (NPA), response for advice (RAD)
Iterative-based	- Iterative metrics are a type of software maintenance metric that track the progress and effectiveness of the maintenance process over multiple iterations or cycles. - These metrics provide insights into the performance and efficiency of the maintenance activities	- The effectiveness of iterative maintenance metrics depends on the quality and accuracy of the data collection and analysed. - The iterative nature may require significant resources and effort.	Mean Lead Time for Changes, Mean Time to Recover, Change Failure Rate, and Deployment Frequency.

In modern development environments like DevOps, iterative-based metrics have become more appealing with the utilization of data becoming crucial [45]. These metrics are valuable as they monitor the effectiveness of maintenance activities over multiple iterations or cycles, helping organizations identify recurring issues and bottlenecks in their development processes. While traditional metrics like lines of code (LOC) and function points have been widely used, they may not provide meaningful insights into software maintainability in fast-paced, dynamic environments. More relevant metrics include the average time required for a change (e.g., a new feature or a bug

fix) and the percentage of changes that lead to failures or incidents. These metrics offer deeper insights into the system's ability to maintain stability while evolving.

However, relying solely on these iterative metrics may not fully capture the overall maintainability of a software system. For a more comprehensive understanding, they should be used alongside other metrics, such as object-oriented, component-based, and aspect-oriented metrics. These approaches can reveal valuable insights into software quality, maintainability, and the effectiveness of the development process. Additionally, incorporating tool-based approaches can significantly aid in making more informed decisions to improve software maintainability [11,18].

Based on the findings and evolution of software maintainability metrics, future directions should focus on the development of context-aware metrics tailored specifically for DevOps and Agile environments. Given that these environments emphasize rapid iteration and continuous deployment, it is essential to create metrics that address the unique maintainability challenges posed by such high-frequency, dynamic workflows. Context-aware metrics, which consider factors such as deployment frequency, code churn, and incident response times, can provide a more accurate measure of software maintainability and resilience in these settings.

Moreover, an emerging trend in DevOps research is the focus on microservices architecture, which has become a significant paradigm in modern development practices. As software architectures evolve, maintainability metrics must adapt to newer paradigms like microservices, cloud-native, and serverless computing. These architectures introduce distinct challenges in modularity, scalability, and inter-service communication. Future research could explore tailored metrics for assessing maintainability in these contexts, particularly in areas such as inter-service dependency management, fault tolerance, and scalability. These emerging metrics would be crucial in ensuring that software systems built on these new architectures maintain their quality and adaptability as they evolve.

5. Conclusion

Software maintenance is a crucial part of the software development process, which includes efforts targeted at preserving, upgrading, and improving software systems after they have been deployed. It is one of the most important aspects when evaluating the quality of a software product, and it should be considered and focused on to avoid poor system performance and long-term viability. This paper discusses the established metrics for software maintenance, their limitations, and the need for continuous related metrics to address the challenges of modern software development practices. Iterative-based metrics are a type of software maintenance metric that track the progress and effectiveness of the maintenance process over multiple iterations or cycles. Unlike traditional metrics that focus on the maintainability aspects of a software system, iterative metrics contributing to the overall goal of continuous maintenance improvement. These metrics are particularly relevant in modern development environments like DevOps, where the utilization of data has become crucial for continuous improvement and delivery. One of the potential future avenues is to conduct empirical studies that compare the performance of different maintainability metrics in DevOps settings.

Acknowledgement

This research is funded by the Talent and Publications Enhancement Research Grant (TAPERG/2023/UMT/2223) titled "Empirical Analysis of Software Maintainability Metrics in DevOps Environments" at Universiti Malaysia Terengganu. The authors would also like to express their gratitude to the undergraduate students enrolled in CSE3513 Software Maintenance and Evolution

at Universiti Malaysia Terengganu (UMT) during the 2023 academic session for their valuable contributions to the literature sources used in this article.

References

- [1] Coelho, Jailton, Marco Tulio Valente, Luciano Milen, and Luciana L. Silva. "Is this GitHub project maintained? Measuring the level of maintenance activity of open-source projects." *Information and Software Technology* 122 (2020): 106274. <https://doi.org/10.1016/j.infsof.2020.106274>
- [2] S Chowdhury, Shaiful, Reid Holmes, Andy Zaidman, and Rick Kazman. "Revisiting the debate: Are code metrics useful for measuring maintenance effort?." *Empirical Software Engineering* 27, no. 6 (2022): 158. <https://doi.org/10.1007/s10664-022-10193-8>
- [3] Sivapurnima, S., and D. Manjula. "Adaptive Deep Learning Model for Software Bug Detection and Classification." *Computer Systems Science & Engineering* 45, no. 2 (2023). <https://doi.org/10.32604/csse.2023.025991>
- [4] A Trautsch, Alexander, Johannes Erbel, Steffen Herbold, and Jens Grabowski. "What really changes when developers intend to improve their source code: a commit-level study of static metric value and static analysis warning changes." *Empirical Software Engineering* 28, no. 2 (2023): 30. <https://doi.org/10.1007/s10664-022-10257-9>
- [5] "ISO/IEC 25010 Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models." (2023).
- [6] Ahmad, Fauziah Sh, Ong Sin Ru, Bhavani Kolan de Velu, Sya Azmeela Shariff, and Aini Suzanna Ariffin. "Nurturing Engineers of Industry 4.0 Workforce with Entrepreneurial Quality Skills in Leadership, Innovation, Communication, and Teamwork." *Journal of Advanced Research in Technology and Innovation Management* 5, no. 1 (2022): 1-11.
- [7] Azamuddin, WMH Ahmad, Z. A. M. Noh, A. S. H. Basari, and A. S. H. Shibghatullah. "Enhancement Quality of Service at ADTECBP LAN Using Traffic Policing." *Journal of Advanced Research in Computing and Applications* 4, no. 1 (2016): 9-19.
- [8] Coleman, Don, Dan Ash, Bruce Lowther, and Paul Oman. "Using metrics to evaluate software system maintainability." *Computer* 27, no. 8 (1994): 44-49. <https://doi.org/10.1109/2.303623>
- [9] Ouellet, Alexandre, and Mourad Badri. "Combining object-oriented metrics and centrality measures to predict faults in object-oriented software: An empirical validation." *Journal of Software: Evolution and Process* 36, no. 4 (2024): e2548. <https://doi.org/10.1002/smr.2548>
- [10] Laukkanen, Eero, Juha Itkonen, and Casper Lassenius. "Problems, causes and solutions when adopting continuous delivery—A systematic literature review." *Information and Software Technology* 82 (2017): 55-79. <https://doi.org/10.1016/j.infsof.2016.10.001>
- [11] Almashhadani, Mohammed, Alok Mishra, Ali Yazici, and Muhammad Younas. "Challenges in agile software maintenance for local and global development: An empirical assessment." *Information* 14, no. 5 (2023): 261. <https://doi.org/10.3390/info14050261>
- [12] Lwakatare, Lucy Ellen, Terhi Kilamo, Teemu Karvonen, Tanja Sauvola, Ville Heikkilä, Juha Itkonen, Pasi Kuvaja, Tommi Mikkonen, Markku Oivo, and Casper Lassenius. "DevOps in practice: A multiple case study of five companies." *Information and software technology* 114 (2019): 217-230. <https://doi.org/10.1016/j.infsof.2019.06.010>
- [13] Grande, Rubén, Aurora Vizcaíno, and Félix O. García. "Is it worth adopting DevOps practices in Global Software Engineering? Possible challenges and benefits." *Computer Standards & Interfaces* 87 (2024): 103767. <https://doi.org/10.1016/j.csi.2023.103767>
- [14] Lewis, John, and Sallie Henry. "A methodology for integrating maintainability using software metrics." In *Proceedings. Conference on Software Maintenance-1989*, pp. 32-39. IEEE, 1989. <https://doi.org/10.1109/ICSM.1989.65191>
- [15] Bagheri, Ebrahim, and Dragan Gasevic. "Assessing the maintainability of software product line feature models using structural metrics." *Software Quality Journal* 19 (2011): 579-612. <https://doi.org/10.1007/s11219-010-9127-2>
- [16] Basili, Victor R., Lionel C. Briand, and Walcélio L. Melo. "A validation of object-oriented design metrics as quality indicators." *IEEE Transactions on software engineering* 22, no. 10 (1996): 751-761. <https://doi.org/10.1109/32.544352>
- [17] de AG Saraiva, Juliana, Micael S. De França, Sérgio CB Soares, J. C. L. Fernando Filho, and Renata MCR de Souza. "Classifying metrics for assessing object-oriented software maintainability: A family of metrics' catalogs." *Journal of Systems and Software* 103 (2015): 85-101. <https://doi.org/10.1016/j.jss.2015.01.014>

- [18] Ardito, Luca, Riccardo Coppola, Luca Barbato, and Diego Verga. "A Tool-Based Perspective on Software Code Maintainability Metrics: A Systematic Literature Review." *Scientific Programming* 2020, no. 1 (2020): 8840389. <https://doi.org/10.1155/2020/8840389>
- [19] Ebert, Christof, James Cain, Giuliano Antoniol, Steve Counsell, and Phillip Laplante. "Cyclomatic complexity." *IEEE software* 33, no. 6 (2016): 27-29. <https://doi.org/10.1109/MS.2016.147>
- [20] Nagappan, Nachiappan, and Thomas Ball. "Use of relative code churn measures to predict system defect density." In *Proceedings of the 27th international conference on Software engineering*, pp. 284-292. 2005. <https://doi.org/10.1145/1062455.1062514>
- [21] Henry, Sallie, and Dennis Kafura. "Software structure metrics based on information flow." *IEEE transactions on Software Engineering* 5 (1981): 510-518. <https://doi.org/10.1109/TSE.1981.231113>
- [22] Oman, Paul, and Jack Hagemester. "Metrics for assessing a software system's maintainability." In *Proceedings Conference on Software Maintenance 1992*, pp. 337-338. IEEE Computer Society, 1992. <https://doi.org/10.1109/ICSM.1992.242525>
- [23] Chidamber, Shyam R., and Chris F. Kemerer. "A metrics suite for object oriented design." *IEEE Transactions on software engineering* 20, no. 6 (1994): 476-493. <https://doi.org/10.1109/32.295895>
- [24] Ganpati, Anita, Arvind Kalia, and Hardeep Singh. "A comparative study of maintainability index of open source software." *Int. J. Emerg. Technol. Adv. Eng* 2, no. 10 (2012): 228-230.
- [25] Cusumano, Michael A., and Richard W. Selby. *Microsoft secrets: how the world's most powerful software company creates technology, shapes markets, and manages people*. Simon and Schuster, 1998.
- [26] Syed-Mohamad, Sharifah Mashita, and Nur Shazwani Md Akhir. "SoReady: an extension of the test and defect coverage-based analytics model for pull-based software development." In *2019 26th Asia-Pacific Software Engineering Conference (APSEC)*, pp. 9-14. IEEE, 2019. <https://doi.org/10.1109/APSEC48747.2019.00011>
- [27] Lai, Sen-Tarng, Heru Susanto, and Fang-Yie Leu. "Combining Pipeline Quality with Automation to CI/CD Process for Improving Quality and Efficiency of Software Maintenance." In *International Conference on Innovative Mobile and Internet Services in Ubiquitous Computing*, pp. 362-372. Cham: Springer International Publishing, 2021. https://doi.org/10.1007/978-3-030-79728-7_36
- [28] Pang, Candy, and Abram Hindle. "Continuous maintenance." In *2016 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pp. 458-462. IEEE, 2016. <https://doi.org/10.1109/ICSME.2016.45>
- [29] Roy, Rajkumar, Rainer Stark, Kirsten Tracht, Shozo Takata, and Masahiko Mori. "Continuous maintenance and the future—Foundations and technological challenges." *Cirp Annals* 65, no. 2 (2016): 667-688. <https://doi.org/10.1016/j.cirp.2016.06.006>
- [30] Azad, Nasreen, and Sami Hyrnsalmi. "DevOps critical success factors—A systematic literature review." *Information and Software Technology* 157 (2023): 107150. <https://doi.org/10.1016/j.infsof.2023.107150>
- [31] Bermon Angarita, Leonardo, Alvaro Fernández Del Carpio, and Andrés Alberto Osorio Londoño. "A Bibliometric Analysis of DevOps Metrics." *DESIDOC Journal of Library & Information Technology* 42, no. 6 (2022). <https://doi.org/10.14429/djlit.42.6.18365>
- [32] Zimmerer, Peter. "Strategy for continuous testing in iDevOps." In *Proceedings of the 40th International Conference on Software Engineering: Companion Proceedings*, pp. 532-533. 2018. <https://doi.org/10.1145/3183440.3183465>
- [33] Van Belzen, Michiel, Jos Trienekens, and Rob Kusters. "Critical success factors of continuous practices in a DevOps context." (2019).
- [34] Akbar, Muhammad Azeem, Arif Ali Khan, Najmul Islam, and Sajjad Mahmood. "DevOps project management success factors: A decision-making framework." *Software: Practice and Experience* 54, no. 2 (2024): 257-280. <https://doi.org/10.1002/spe.3269>
- [35] Muñoz, Mirna, Mario Negrete, and Jezreel Mejía. "Proposal to avoid issues in the devops implementation: A systematic literature review." *New Knowledge in Information Systems and Technologies: Volume 1* (2019): 666-677. https://doi.org/10.1007/978-3-030-16181-1_63
- [36] Albuquerque, Adriano Bessa, and Vinicius Lima Cruz. "Implementing DevOps in legacy systems." In *Intelligent Systems in Cybernetics and Automation Control Theory 2*, pp. 143-161. Springer International Publishing, 2019. https://doi.org/10.1007/978-3-030-00184-1_14
- [37] Morales, Jose Andre, Hasan Yasar, and Aaron Volkman. "Implementing DevOps practices in highly regulated environments." In *Proceedings of the 19th International Conference on Agile Software Development: Companion*, pp. 1-9. 2018. <https://doi.org/10.1145/3234152.3234188>
- [38] Wiedemann, Anna, Manuel Wiesche, Heiko Gewald, and Helmut Krcmar. "Implementing the planning process within DevOps teams to achieve continuous innovation." (2019). <https://doi.org/10.24251/HICSS.2019.841>
- [39] Altunel, Haluk, and Bilge Say. "Software product system model: a customer-value oriented, adaptable, devops-based product model." *SN Computer Science* 3, no. 1 (2022): 38. <https://doi.org/10.1007/s42979-021-00899-9>

- [40] Céspedes, Daniel, Paula Angeleri, Karin Melendez, and Abraham Dávila. "Software product quality in DevOps contexts: a systematic literature review." In *Trends and Applications in Software Engineering: Proceedings of the 8th International Conference on Software Process Improvement (CIMPS 2019)*, pp. 51-64. Springer International Publishing, 2020. https://doi.org/10.1007/978-3-030-33547-2_5
- [41] Gupta, Rajeev Kumar, Mekanathan Venkatachalapathy, and Feroze Khan Jeberla. "Challenges in adopting continuous delivery and DevOps in a globally distributed product team: A case study of a healthcare organization." In *2019 ACM/IEEE 14th International Conference on Global Software Engineering (ICGSE)*, pp. 30-34. IEEE, 2019. <https://doi.org/10.1109/ICGSE.2019.00020>
- [42] Chhillar, Rajender Singh, and Sonal Gahlot. "An evolution of software metrics: a review." In *Proceedings of the International Conference on Advances in Image Processing*, pp. 139-143. 2017. <https://doi.org/10.1145/3133264.3133297>
- [43] Gary Haines, David Carney, and John Foreman, *Component-Based Software Development/COTS Integration*. Software Engineering Institute (SEI), Carnegie Mellon University, 2007.
- [44] Ghareb, Mazen Ismaeel, and Gary Allen. "State of the art metrics for aspect oriented programming." In *AIP conference proceedings*, vol. 1952, no. 1. AIP Publishing, 2018. <https://doi.org/10.1063/1.5032069>
- [45] Gradišnik, Mitja, Tina Beranič, and Sašo Karakatič. "Impact of historical software metric changes in predicting future maintainability trends in open-source software development." *Applied Sciences* 10, no. 13 (2020): 4624. <https://doi.org/10.3390/app10134624>